Properties of A*

- Complete? Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes

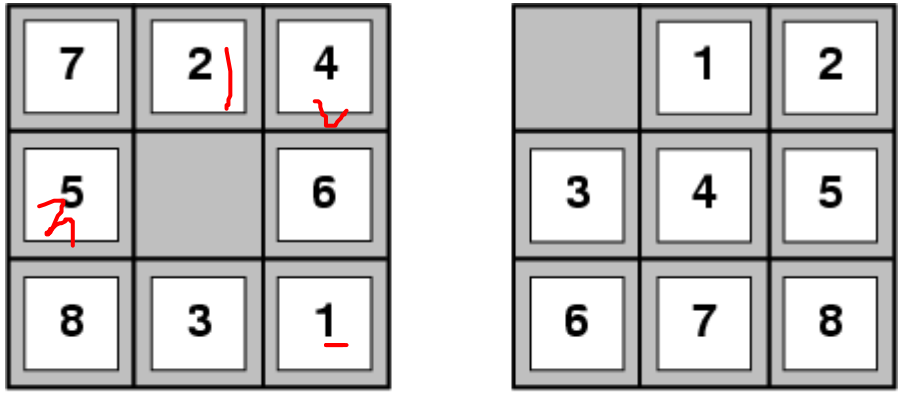
$f(n) = g(n) + h(n)$

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)



Start State Goal State

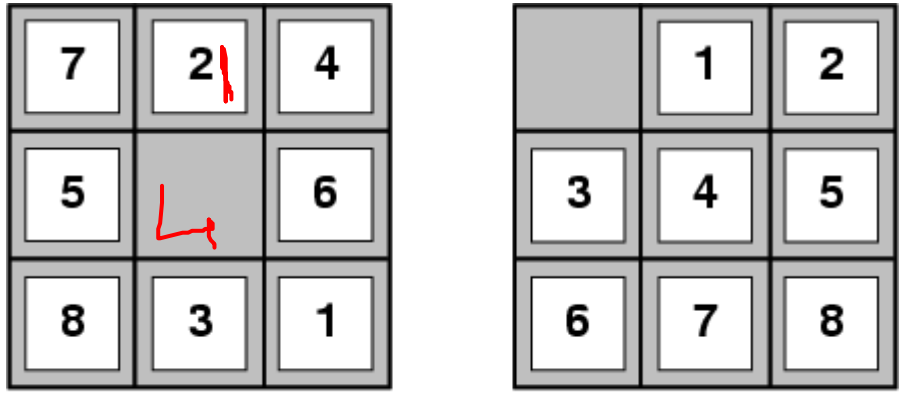
- $h_1(S) = ?$
- $h_2(S) = ?$

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)



Start State Goal State

- $h_1(S)$ = ? 8
- $h_2(S)$ = ? 3+1+2+2+2+3+3+2 = 18

Relaxed problems

- A problem with fewer restrictions on the actions is called a relaxed problem
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
  - The relaxed problem is usually easier to solve optimally and quickly.
  - if you have a relaxed problem that you can solve optimally and you find the cost of the best solution for that problem, you can use that cost as a heuristic estimate for the original, more complex problem.
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

Artificial Intelligence

Lecture:7 Local Search Algorithm

Today’s Topic

- Informed Search Strategies: one that uses problem specific knowledge
  - Heuristic Functions
  - Local Search and Optimization

Local search algorithms

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n- queens
  - what matters is the final configuration of queens, not the order in which they are added.
- In such cases, we can use local search algorithms
- keep a single "current" state, try to improve it

Local search algorithms

- Local search algorithms work by iteratively improving the candidate solution by making small changes and evaluating the objective function after each change.

- The objective function helps guide the search process by determining whether a given solution is better or worse than the current solution and helps determine the direction of the search.

- The algorithm continues this process until it reaches a satisfactory solution, or it determines that it cannot find a better solution.

Local Search Algorithms

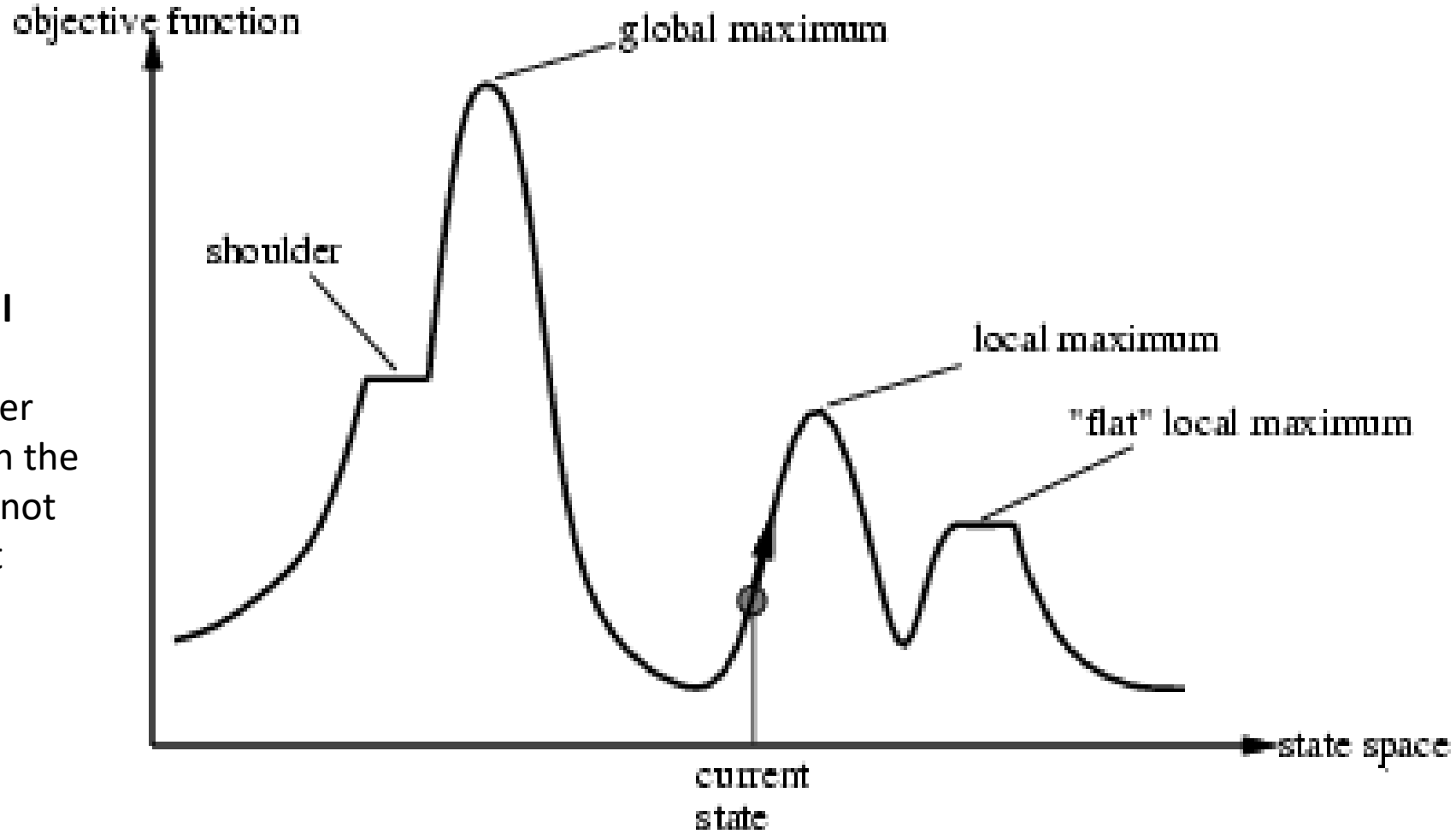
- Advantages:
  - Use very little memory
  - Find reasonable solution in infinite(Continuous) State Space for which systematic algorithms are unsuitable

Local Search Algorithms are useful for Optimization Problems, in which aim is to find best state according to an **Objective Function**

Local Search Algorithm

- Problem: depending on initial state, can get stuck in local maxima

In the context of optimization, **a local maxima** refers to a solution that is better than its neighbors in the solution space, but not necessarily the best possible solution.



A complete **local search algorithm** always finds a goal if one exists; an optimal algorithm always finds a global minimum/maximum

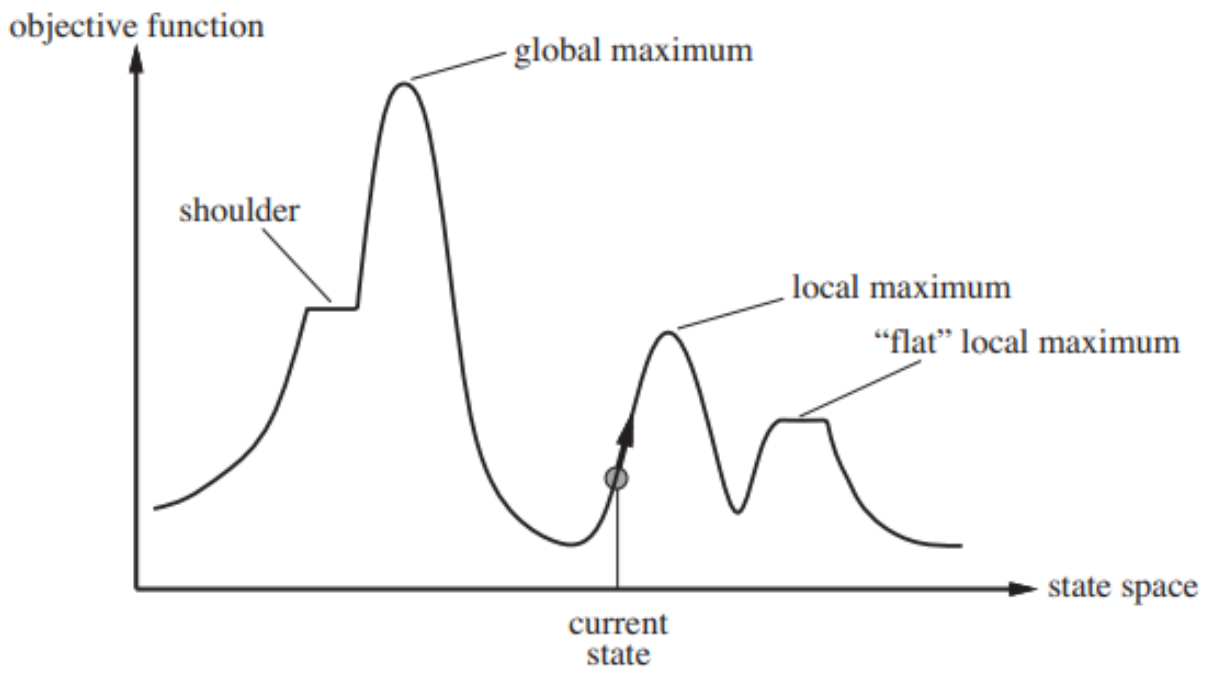


Figure 4.1 A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. Hill-climbing search modifies the current state to try to improve it, as shown by the arrow. The various topographic features are defined in the text.

Hill-climbing search

- "Like climbing Everest in thick fog with amnesia"

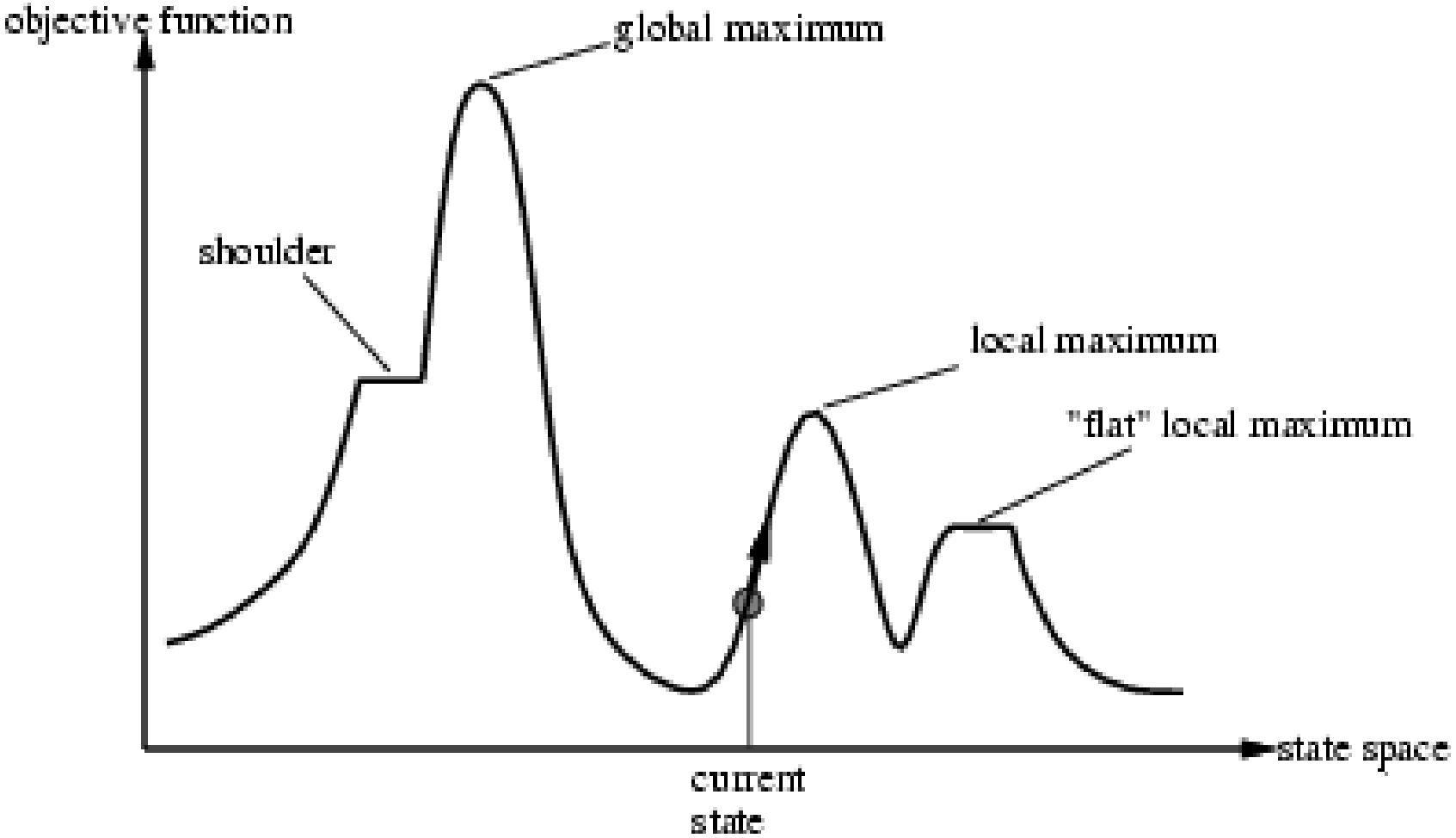
```
function HILL-CLIMBING(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
```

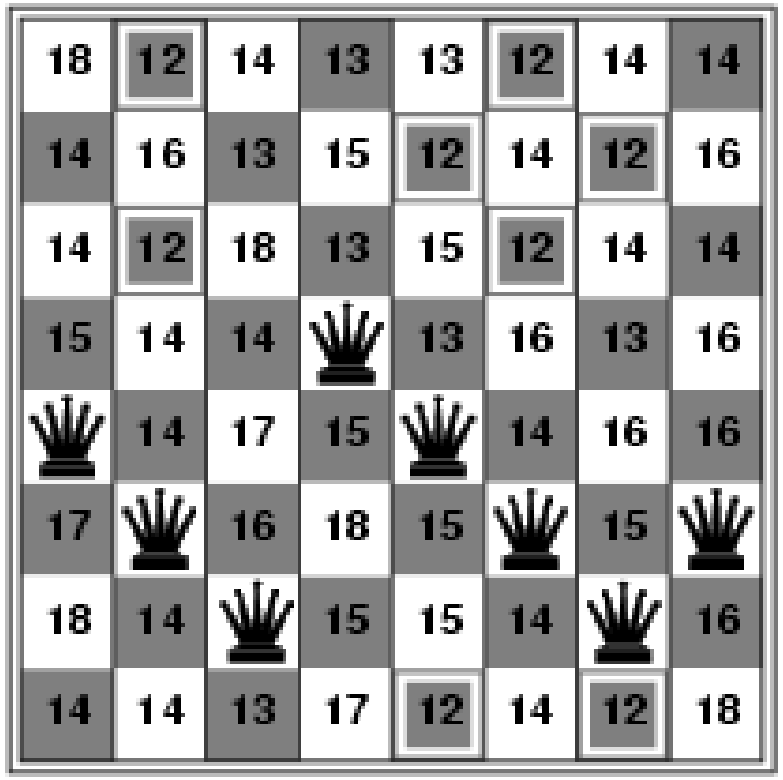
Hill climbing does not look ahead beyond the immediate neighbors of the current state.

Hill-climbing search

- "Like climbing Everest in thick fog with amnesia"

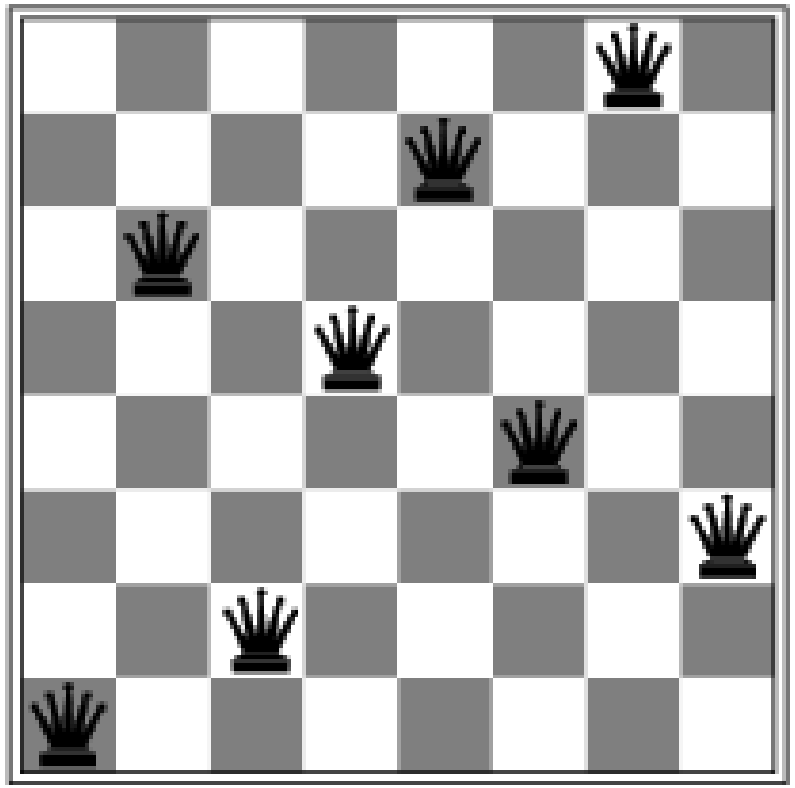


Hill-climbing search: 8-queens problem



- *h* = number of pairs of queens that are attacking each other, either directly or indirectly

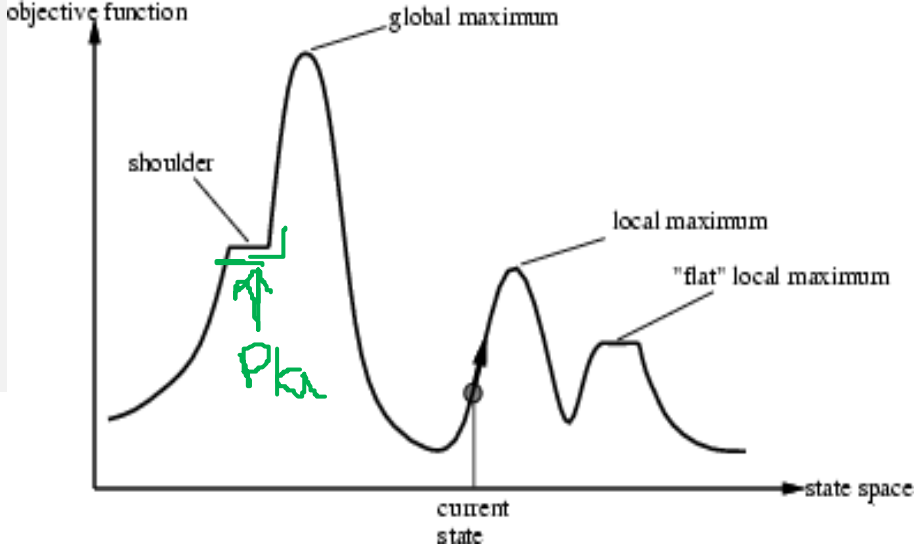
Hill-climbing search: 8-queens problem



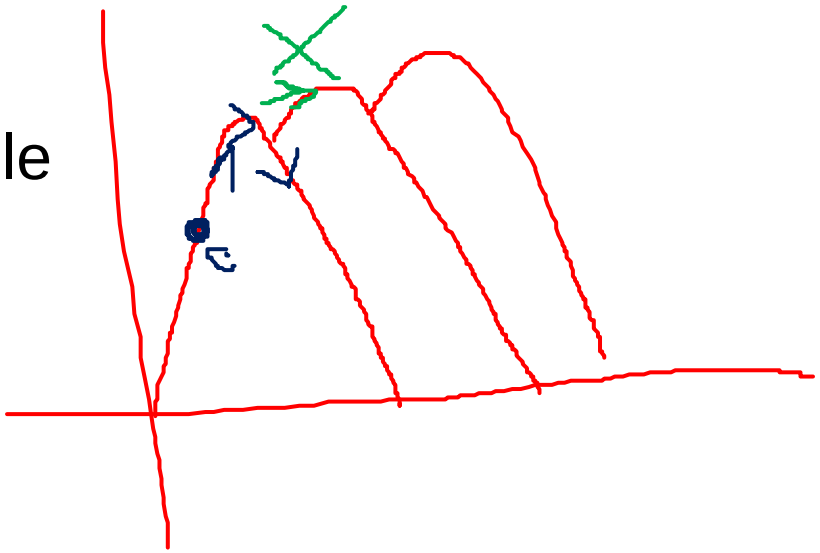
Have moved 5 queens (within their column) and reached close to nearest solution where only one pair of queen is attacking

- A local minimum with $h = 1$

Problems in Hill-climbing search



- Local Maxima
- Plateau/ Flat Maxima
  - It is a flat region of state space where neighboring states have the same value.
  - **Shoulder**: from which progress is possible
- Ridges
  - sequence of local maxima that is very difficult for greedy algorithms to navigate



Problems in Hill Climbing Search

Ridges result in a sequence of local maxima that is very difficult for greedy algorithms to navigate

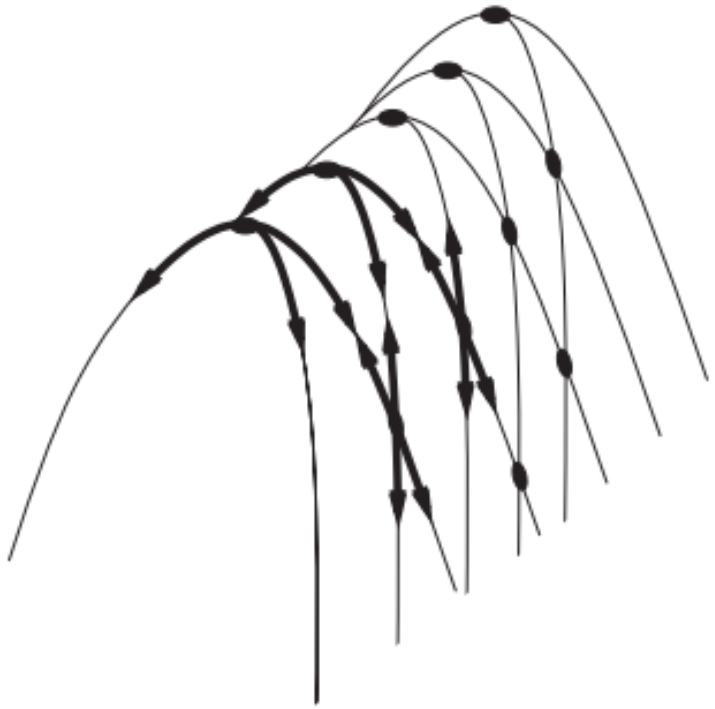


Figure 4.4 Illustration of why ridges cause difficulties for hill climbing. The grid of states (dark circles) is superimposed on a ridge rising from left to right, creating a sequence of local maxima that are not directly connected to each other. From each local maximum, all the available actions point downhill.